

13. Fundamentos de Regresión

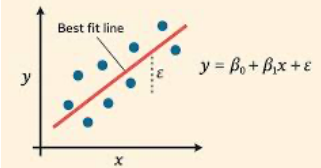
Ciencia de datos para la toma de decisiones I

Jorge Juvenal
Campos
Ferreira

✉ juvenal.campos@tec.mx

¿Qué es la regresión?

LINEAR REGRESSION



Es una herramienta para **predecir un valor numérico (y)** a partir de otras variables conocidas (**X**).

Ejemplo 1: precio de una casa

Conoces sus metros cuadrados, ubicación y antigüedad. La regresión encuentra **la fórmula que relaciona estas variables con el precio**.

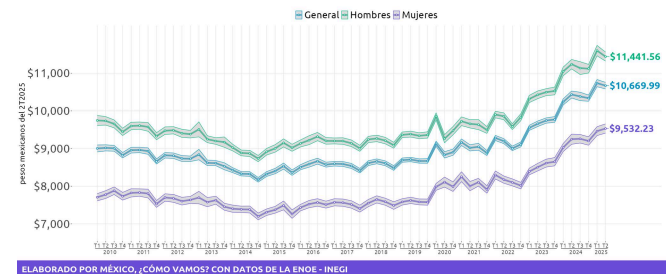


Ejemplo 2: ingreso de un trabajador

Dado el nivel de educación de una persona, la región del país donde trabaja, los años de experiencia, el sector en el que se desempeña, si el trabajo es formal o informal o si el trabajador es hombre o mujer (etc), se puede **estimar el ingreso laboral de una persona**.

Ingreso laboral mensual promedio Por sexo, 2T2025

México
CÓMO VAMOS



Conceptos clave

Variable que queremos predecir = RESPUESTA (Y, dependiente - respuesta)

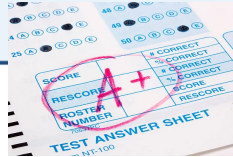
Variables que usamos para predecir = PREDICTORES (X, independientes - predictoras)

La regresión está en TODAS partes

Predecir el
precio de
una casa



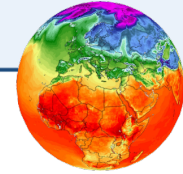
Estimar
calificaciones
escolares



Predecir
ventas de
un producto



Modelar
temperatura
climática



Recomendar
películas en
Netflix



Predecir el
tráfico en
una ciudad



Estimar el
rendimiento
deportivo



La regresión es una de las herramientas **más usadas** en ciencia de datos.

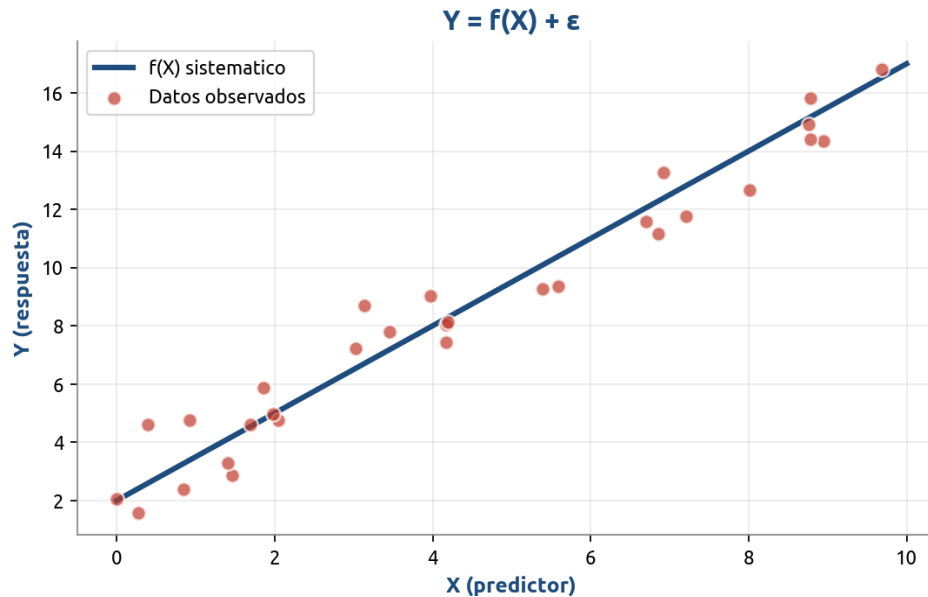
En el caso de la regresión lineal, ayuda mucho su **simplicidad** y su **explicabilidad**.

El planteamiento general

Queremos encontrar la relación entre la variable respuesta **y** y las variables explicativas **X**. Asumimos que existe una relación entre X e Y, pero con algo de ruido aleatorio (ϵ):

$$Y = f(X) + \epsilon$$

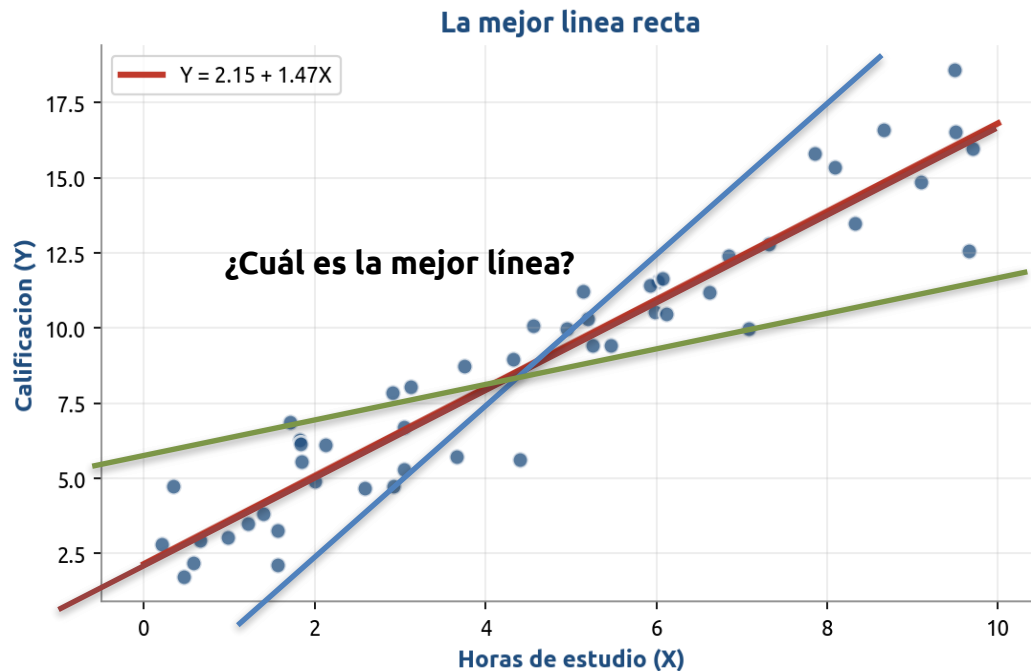
f(X) es la fórmula o función de esa · ϵ es el error aleatorio



Nuestro objetivo: estimar $f(X)$ usando los datos disponibles. El error ϵ no se puede eliminar, pero sí cuantificar.

Regresión lineal simple

La idea más sencilla: trazar una línea recta a través de los datos.



$$Y = \beta_0 + \beta_1 X$$

Fórmula de la recta: intercepto · pendiente

¿Qué es β_0 ?

El intercepto: valor de Y cuando X = 0.
Donde la línea cruza el eje vertical.

¿Qué es β_1 ?

La pendiente: cuánto cambia Y al aumentar X en 1 unidad.
Si $\beta_1 = 1.5$, cada hora extra de estudio sube 1.5 puntos.

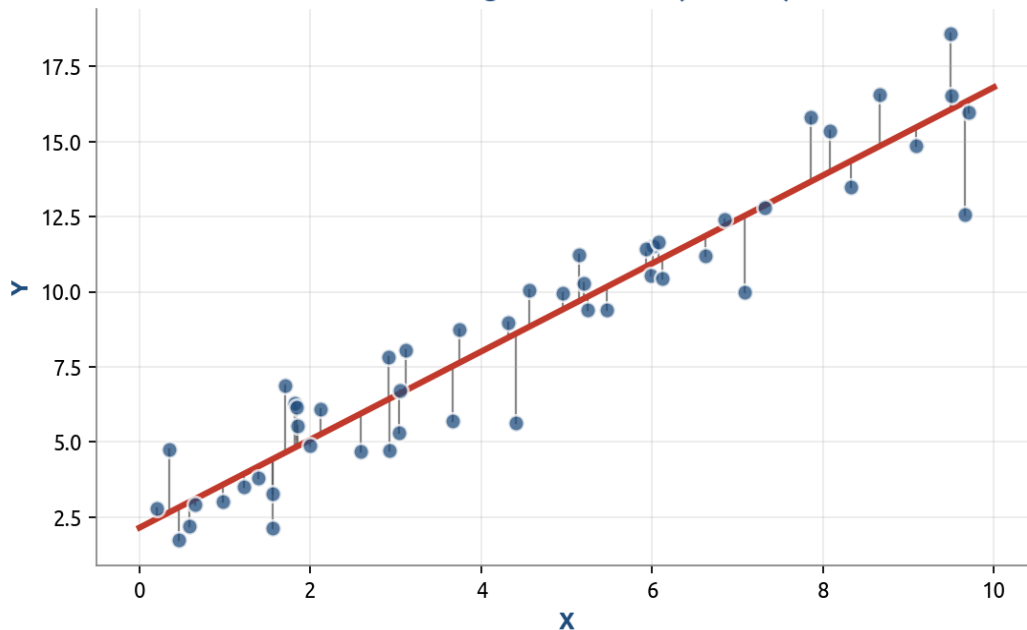
¿Lineal en qué sentido?

"Lineal" se refiere a los parámetros β , no necesariamente a X. Podemos incluir X^2 o $\log(X)$ y seguir teniendo un modelo lineal.

¿Cómo encontrar la mejor línea?

El método de mínimos cuadrados (OLS): minimizar la suma de errores al cuadrado.

Cada línea gris = un error (residuo)



1. Medir errores

Para cada punto, distancia vertical entre lo real y lo predicho.

2. Elevar al cuadrado

Así todos los errores son positivos y los grandes pesan más.

3. Minimizar la suma

La mejor línea es la que MINIMIZA la suma de errores al cuadrado (RSS).

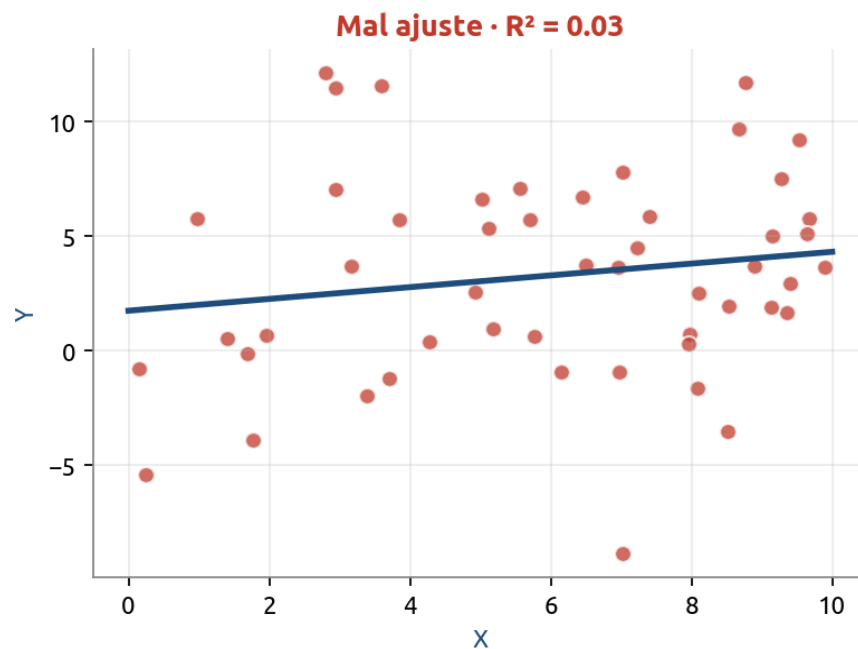
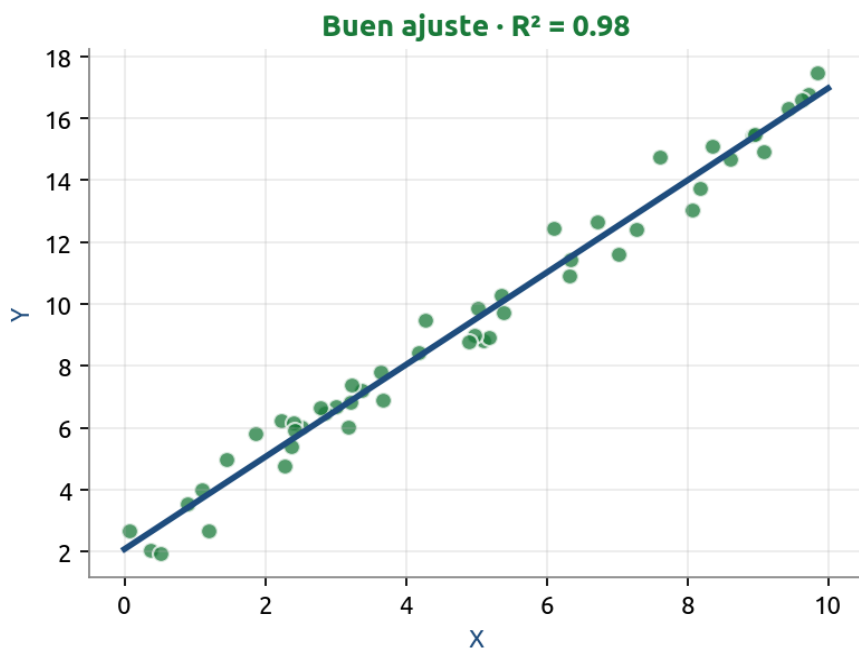
$$RSS = \sum (y_i - \hat{y}_i)^2$$

Suma de residuos al Cuadrado

Esto se llama OLS (Ordinary Least Squares) — Mínimos Cuadrados Ordinarios.

¿Qué tan buena es nuestra línea?

El indicador estrella: R^2



R^2 (coeficiente de determinación)

Va de 0 a 1. Indica el porcentaje de variación en Y explicada por el modelo.

$R^2 = 0.95 \Rightarrow 95\%$ de la variabilidad explicada.

RSE (error estándar residual)

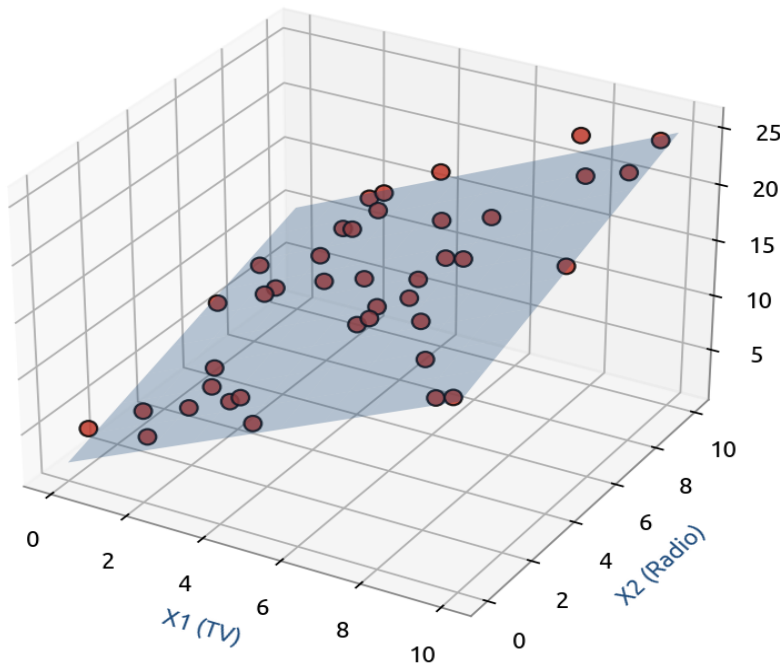
Promedio de cuánto se desvían las predicciones de los valores reales.

Está en las mismas unidades que Y (más interpretable).

Regresión lineal múltiple

Cuando tenemos varios predictores $X_1, X_2, \dots, X_p$.

Regresión múltiple: ya no es una línea, es un PLANO



$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

cada β_j = efecto de X_j con los demás fijos

Ejemplo: predecir VENTAS

X_1 = inversión en TV

X_2 = inversión en radio

X_3 = inversión en prensa

Cada β_j mide el efecto específico de ese medio sobre las ventas.

Importante

Con 2 predictores ya no es una línea: es un PLANO. Con más predictores, un hiperplano (no se puede dibujar, pero la matemática es la misma).

Variables categóricas y dummies

¿Cómo meto variables como sexo, color o ciudad en la regresión?

Solución: convertir a 0 y 1 (variable dummy)

Original

Persona	Sexo
Ana	Mujer
Luis	Hombre
Pía	Mujer
Juan	Hombre



Con variable dummy

Persona	Es mujer?
Ana	1
Luis	0
Pía	1
Juan	0

Regla general

Para K niveles se crean K-1 dummies.

El nivel sin dummy es la categoría base.

Para más de 2 niveles (ej. ciudades CDMX/Guadalajara/Monterrey)

Crear $K-1 = 2$ dummies.

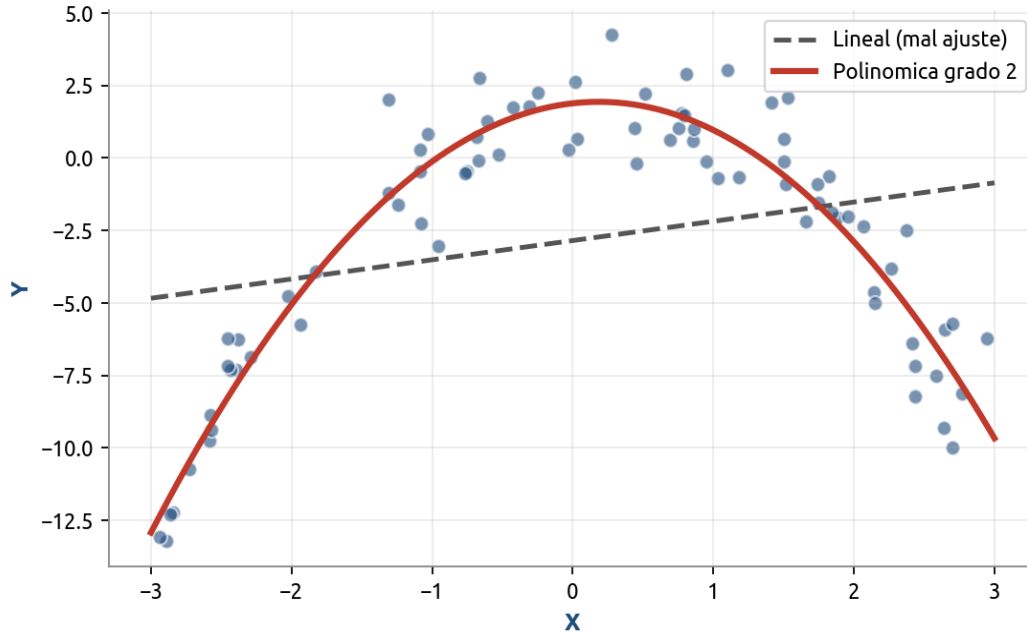
La ciudad faltante es la *ciudad base* (*caso base*).

El R^2 no depende de cuál elijas como base.

Cuando la relación no es recta

Regresión polinómica: añadir potencias de X al modelo.

Cuando la relación no es recta



$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$$

Sigue siendo lineal en los β

Ventaja

Captura curvas suaves sin abandonar ventajas de la regresión lineal.

Cuidado

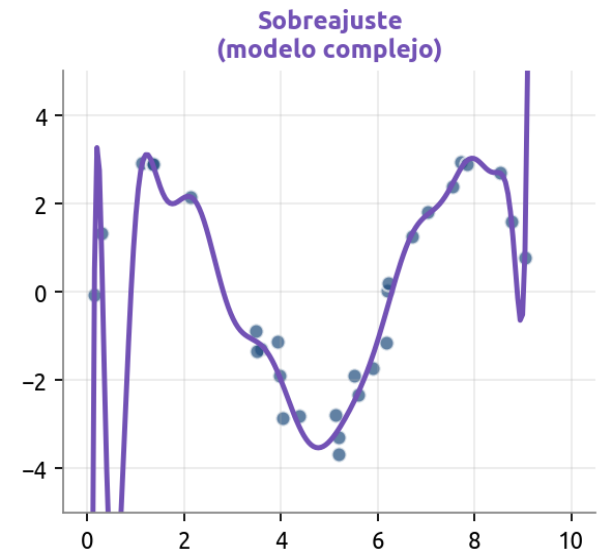
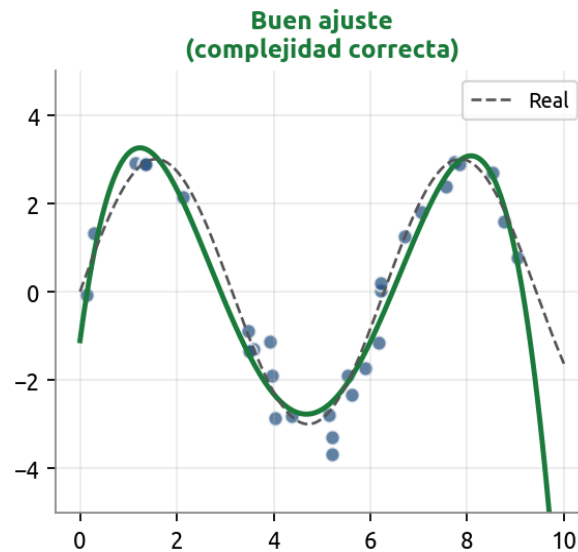
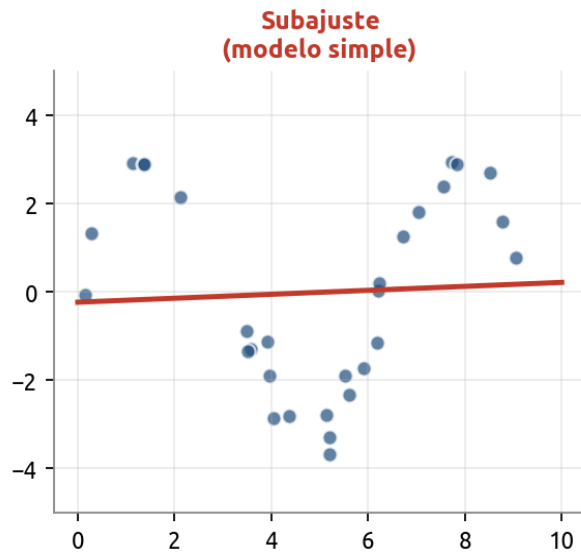
No usar grados muy altos (>3 o 4):
la curva se vuelve muy loca
en los extremos.

Tip práctico

Si necesitas más flexibilidad, hay modelos de regresión más avanzados que no dependen de la forma de la relación de los datos.

El balance sesgo-varianza

El gran dilema de todo modelo: ¿demasiado simple o demasiado complejo?



Subajuste

Modelo demasiado simple.
Alto sesgo, baja varianza.

Buen ajuste

Complejidad correcta.
Equilibrio entre sesgo y
varianza.

Sobreajuste

Modelo demasiado complejo.
Bajo sesgo, alta varianza.

El mejor modelo es el que minimiza el error en datos NUEVOS, no en los de entrenamiento.

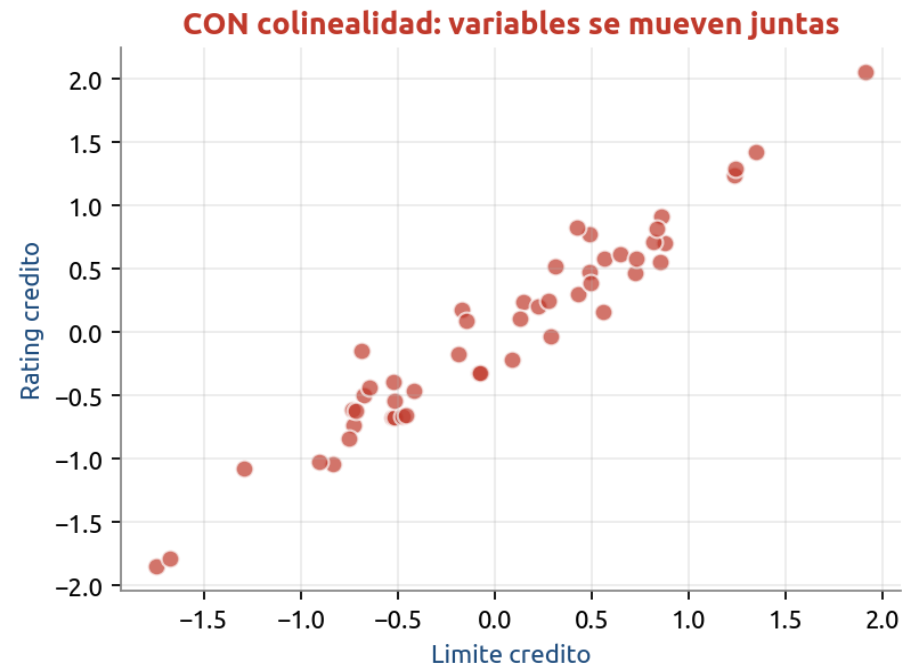
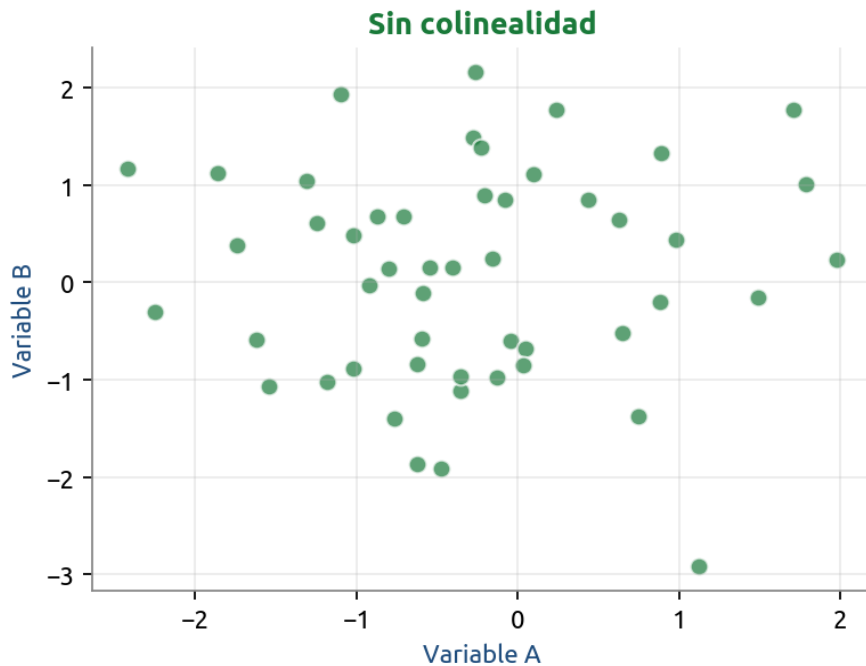
Ejercicio práctico 1

Dados los datos de la carpeta de ejercicios, `home_sales.rds`, realice una regresión lineal utilizando las funciones del paquete `tidymodels`.

1. Cargue los datos del objeto `home_sales.rds`
2. Utilizando `initial_split()`, divida la muestra en **base de entrenamiento** y **base de prueba**.
3. La variable **y** va a ser la variable *selling_price*. Obtenga, para la **base de entrenamiento** y la **base de prueba**, las estadísticas descriptivas de esta variable (min, max, desviación estándar, media). ¿Son diferentes?
4. Defina un modelo de regresión lineal, utilizando el engine “**lm**” y el modo “**regression**”.
5. Tomando como variables **X** a las variables **home_age** y **sqft_living**, ajuste un modelo de regresión sobre la base de entrenamiento. ¿Cuál es la fórmula de la recta de regresión correspondiente?
6. Utilice este modelo con la base de prueba utilizando la función ***predict***. “Péguele” la columna de valores predichos a la base de prueba.
7. Con estos datos en una sola tabla (reales y predichos por el modelo) obtenga el **RMSE** (Raíz del error cuadrático medio) y la **R²**
8. Grafique los valores reales contra los valores predichos.
9. Replique el problema, pero ahora tomando en cuenta todas las variables en **X**.

Problema típico: colinealidad

Cuando dos predictores cuentan básicamente la misma historia.



Cómo detectarla

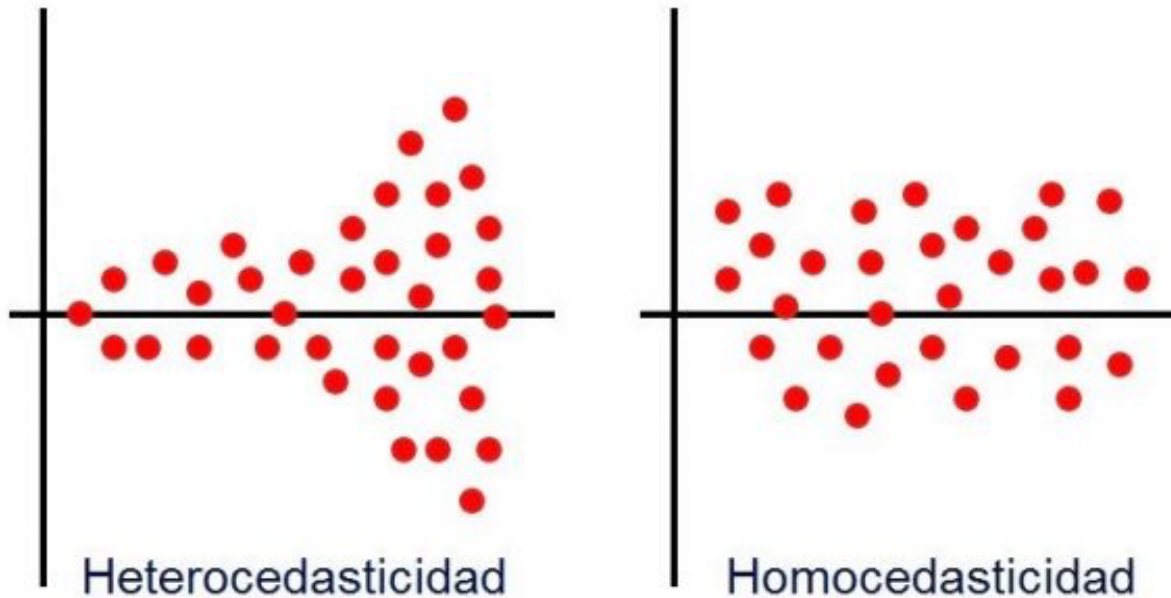
- Matriz de correlación de los predictores
- VIF (Variance Inflation Factor) > 5 o 10

Cómo solucionarla

- Eliminar una de las variables
- Combinar las variables en una sola
- Usar regularización (Ridge)

Problema típico: heteroscedasticidad

Cuando la varianza aumenta conforme aumentan los valores de X .



Cómo detectarla

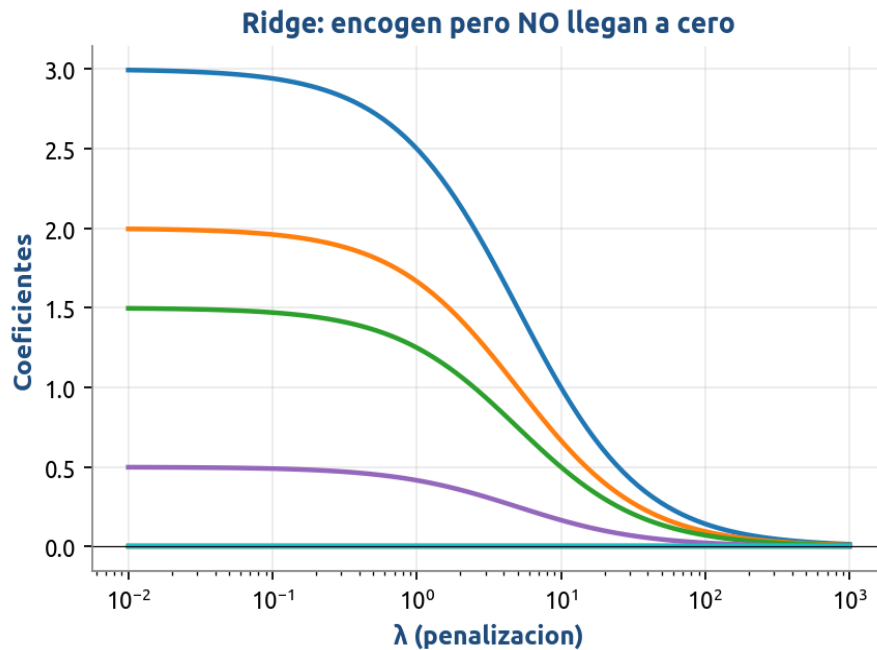
- * Forma típica de embudo (a valores más altos de X , aumenta la varianza)

Cómo solucionarla

- * Transformar y a $\log(y)$

Cuando hay muchas variables: regularización

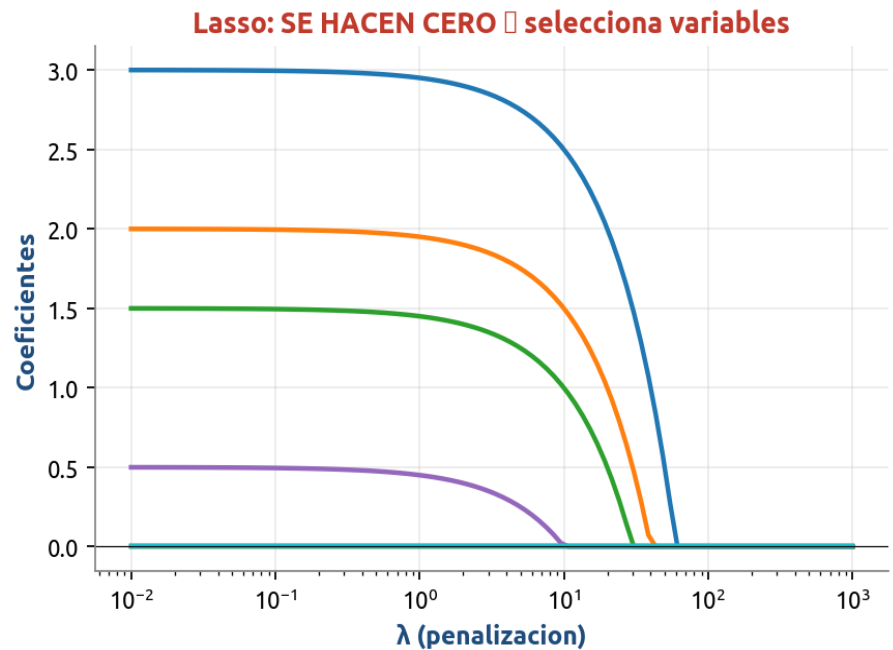
Solución: penalizar coeficientes grandes (encogerlos hacia cero).



Ridge (penalización L2)

$$\text{Loss} = \text{RSS} + \lambda \sum \beta_j^2$$

Encoge coeficientes pero los mantiene todos. Ideal cuando muchos predictores aportan algo.



Lasso (penalización L1)

$$\text{Loss} = \text{RSS} + \lambda \sum |\beta_j|$$

Hace coeficientes EXACTAMENTE cero ⇒ selecciona variables automáticamente.

Ridge vs Lasso: ¿cuál usar?

Elección según la estructura del problema.

RIDGE

¿Cuándo usar Ridge?

- Penalización L2 (cuadrática)
- Encoge coeficientes pero NO los hace cero
- Útil cuando muchos predictores aportan algo
- Mejor para reducir varianza
- Soluciona colinealidad
- Mantiene todas las variables

Mejor cuando todos los predictores son relevantes pero correlacionados.

LASSO

¿Cuándo usar Lasso?

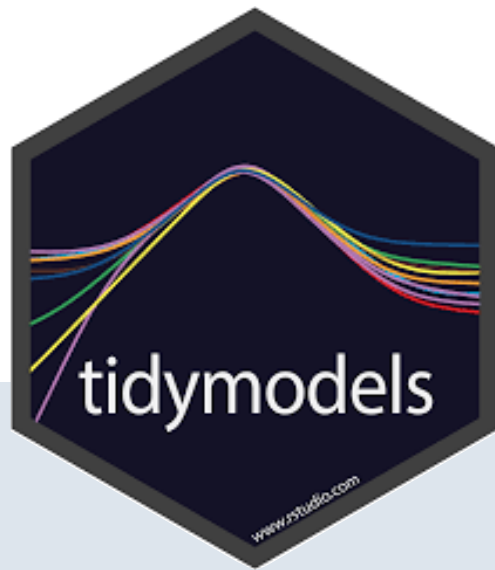
- Penalización L1 (valor absoluto)
- Hace coeficientes EXACTAMENTE 0
- Selecciona variables automáticamente
- Modelos más interpretables
- Útil con muchas variables irrelevantes
- Produce modelos esparsos

Mejor cuando solo unos pocos predictores son verdaderamente relevantes.

¿Qué método elijo?

Guía de decisión rápida según tu situación.

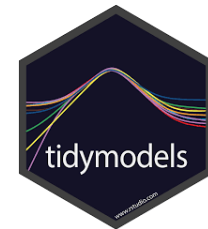
Tu situación	Método recomendado
Pocos predictores, relación lineal	Regresión lineal múltiple
Muchos predictores, todos aportan algo	Ridge
Muchos predictores, solo unos relevantes	Lasso



tidymodels

Implementación práctica en R con un flujo unificado

¿Qué es tidymodels?



Es una colección de paquetes de R diseñados para construir modelos siguiendo los principios del tidyverse.

Antes (caos)

Cada paquete tenía su propia sintaxis:

`lm()`, `glmnet()`, `randomForest()`, `e1071`...

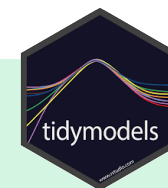
Argumentos distintos, predicciones distintas.

Con tidymodels

Sintaxis unificada y consistente:

`linear_reg()`, `rand_forest()`, `svm_linear()`...

Mismo flujo, mismos verbos, mismas métricas.

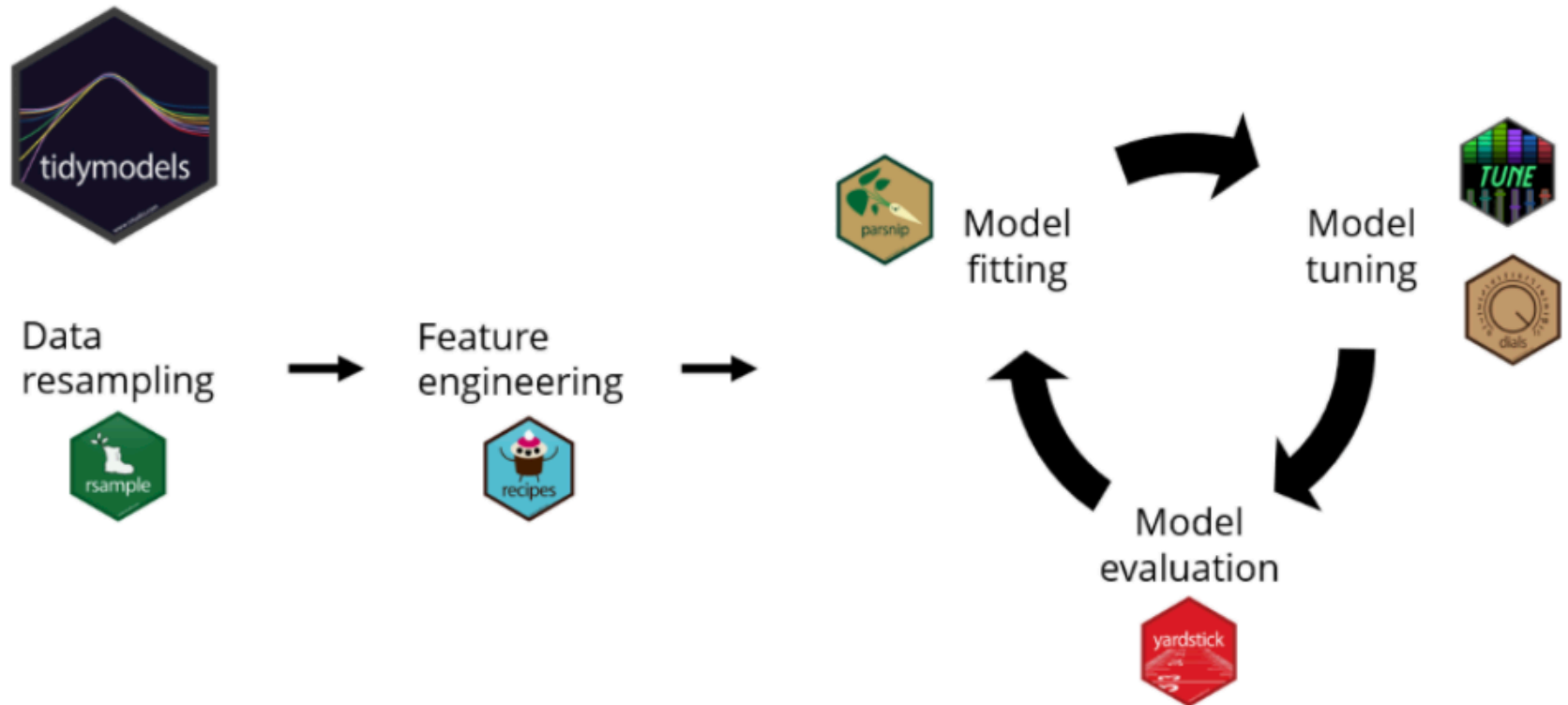


¿Por qué importa?

- Cambias de modelo cambiando una sola línea (`linear_reg` → `rand_forest`).
- Pipelines reproducibles que puedes guardar y compartir.
- Validación cruzada y tuning con un patrón único.
- Compatible con tidyverse: pipas (`%>%`), `dplyr`, `ggplot2`.

```
library(tidymodels)      tidymodels_prefer()
```

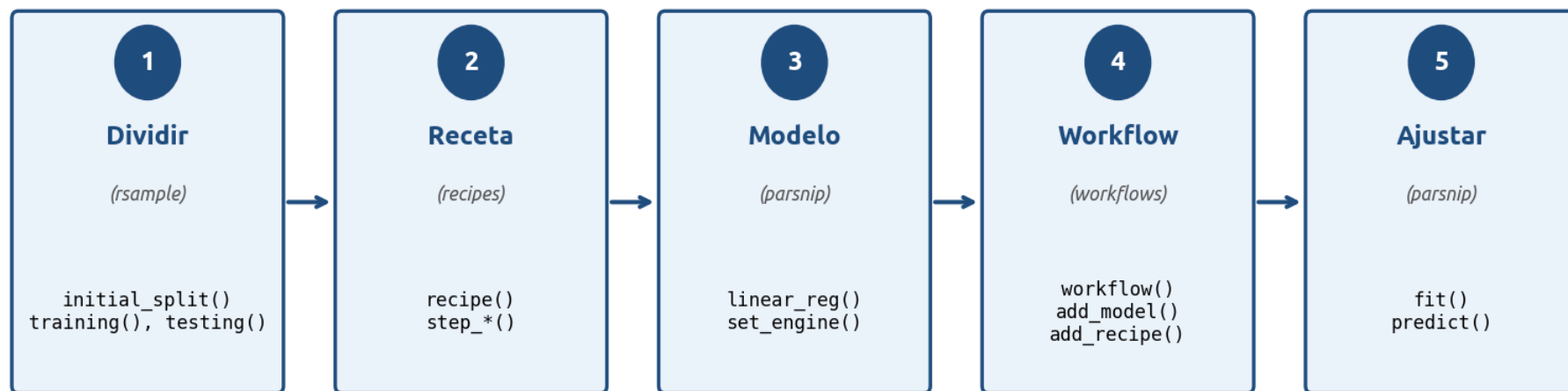
El ecosistema tidymodels



Cada paquete cumple una función en el flujo. Al igual que el **tidyverse** y sus 8 paquetes, al cargar **tidymodels** los cargas todos en la sesión.

Flujo de trabajo estándar

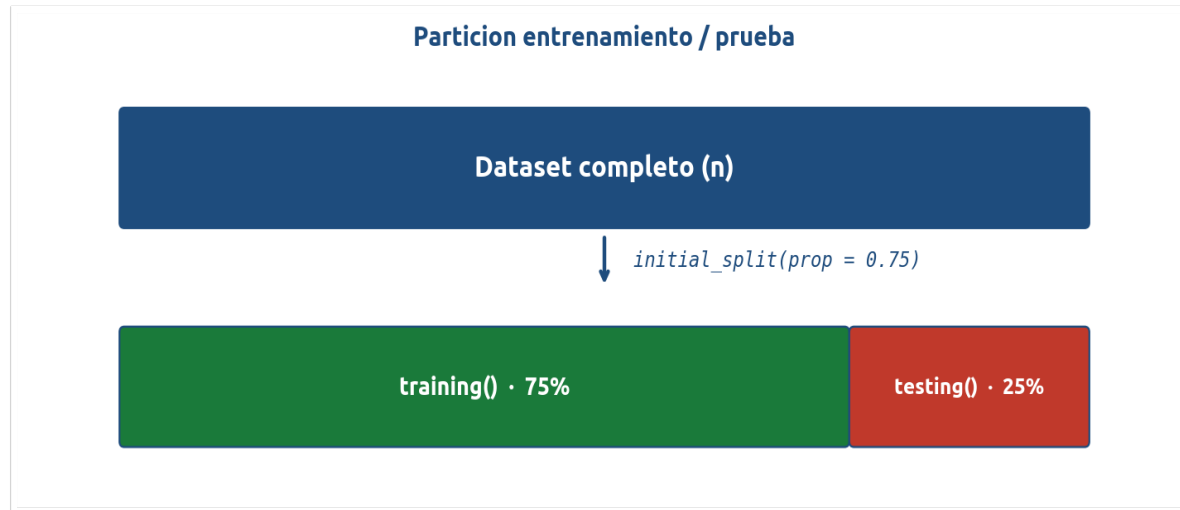
Flujo de trabajo estandar en tidymodels



Patrón mental

"Divido los datos → defino cómo prepararlos → defino el modelo → los uno en un workflow → ajusto y predigo".
Una vez aprendido este flujo, cambiar de modelo solo es cambiar el paso 3.

1. rsample: dividir los datos



```
set.seed(2026)

split_energia <- initial_split(energia, prop = 0.75)
energia_train <- training(split_energia)
energia_test  <- testing(split_energia)
```

initial_split()

Crea el objeto de partición (no separa todavía).
Argumento clave: prop (proporción para entrenamiento).

initial_time_split()

Para series temporales:
las primeras observaciones van a entrenamiento, las últimas a prueba.

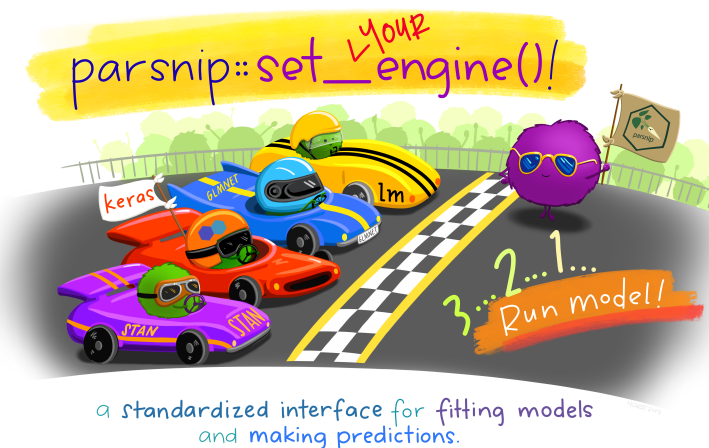
parsnip: especificar el modelo

Un modelo se define con 3 elementos: tipo + motor + modo (regresión/clasificación).

```
# Regresión lineal clásica (OLS)
modelo_lm <- linear_reg() %>%
  set_engine("lm")

# Ridge: penalty = lambda, mixture = 0
modelo_ridge <- linear_reg(penalty = tune(), mixture = 0) %>%
  set_engine("glmnet")

# Lasso: penalty = lambda, mixture = 1
modelo_lasso <- linear_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")
```



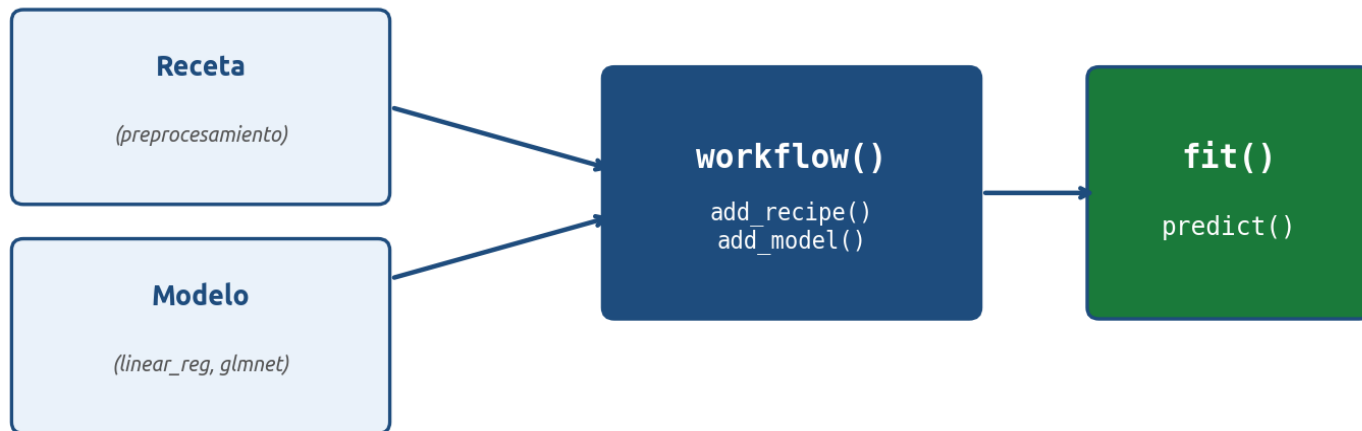
mixture = 0 → Ridge (L2)
mixture = 1 → Lasso (L1)
0 < mixture < 1 → Elastic Net

tune() es un placeholder: indica que ese parámetro se ajustará después por validación cruzada.

Cambiar el modelo solo requiere cambiar la línea de linear_reg(): rand_forest(), boost_tree(), svm_linear() siguen el mismo patrón.

4. workflows: unirlo todo

workflow: receta + modelo en un solo objeto



```
wf_ridge <- workflow() %>%  
  add_model(modelo_ridge) %>%  
  add_recipe(receta_aire)
```

¿Por qué un workflow?

Empaqueta receta + modelo en un solo objeto. Garantiza que el preprocesamiento se aplique igual en train y en test.

5. fit() y predict()

Una vez listo el flujo de trabajo, el ajuste y la predicción son una sola línea cada uno.

```
# Ajustar el modelo
ajuste_base <- fit(wf_base, data = energia_train)

# Predecir en datos de prueba
predicciones <- predict(ajuste_base, new_data = energia_test)

# Combinar predicciones con valores reales
resultados <- predict(ajuste_base, new_data = energia_test) %>%
  bind_cols(energia_test %>% select(consumo_kwh))
```

extract_fit_engine()

Recupera el objeto lm subyacente.
Útil para summary(), anova(), residuos.

tidy()

Convierte coeficientes en un tibble ordenado.
(broom)

```
lm_base <- extract_fit_engine(ajuste_base)
summary(lm_base)
tidy(ajuste_base)
```

6. yardstick: métricas

yardstick calcula métricas de evaluación con sintaxis consistente.

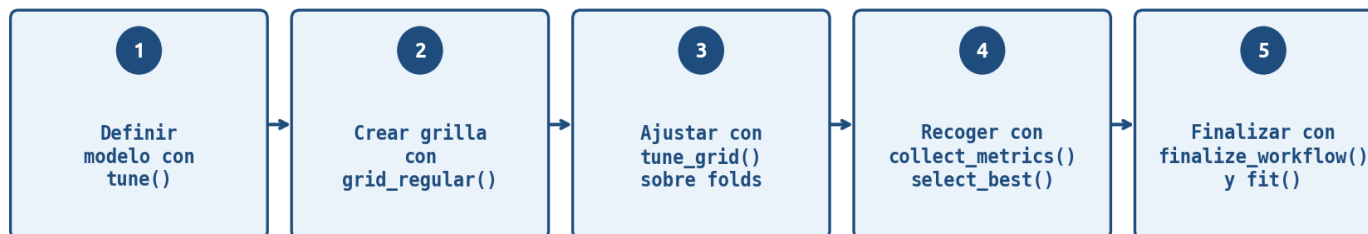
```
# Métricas estándar para regresión
resultados %>%
  metrics(truth = consumo_kwh, estimate = .pred)

# Personalizar el conjunto de métricas
mis_metricas <- metric_set(rmse, rsq, mae)
resultados %>%
  mis_metricas(truth = consumo_kwh, estimate = .pred)
```

Función	Nombre completo	Significado
rmse	Root Mean Squared Error	<i>raíz del error cuadrático medio</i>
rsq	R cuadrado	<i>proporción de varianza explicada</i>
mae	Mean Absolute Error	<i>error absoluto promedio</i>

7. tune: hiperparámetros con validación cruzada

Flujo de tuning de hiperparametros (Ridge / Lasso)



```
# 1. Crear grilla de valores de lambda
grilla_lambda <- grid_regular(penalty(range = c(-3, 1)), levels = 50)

# 2. Ajustar el workflow para cada lambda en cada fold
tune_ridge <- tune_grid(
  wf_ridge, resamples = folds_aire, grid = grilla_lambda,
  metrics = metric_set(rmse, rsq))

# 3. Recoger métricas y elegir el mejor
metricas_cv <- collect_metrics(tune_ridge)
mejor_lambda <- select_best(tune_ridge, metric = "rmse")
```

Ejercicio 1: regresión lineal (energía)

Predecir consumo eléctrico residencial con interacción entre m² y aislamiento.

```
# Receta con interacción
receta_interaccion <- recipe(
  consumo_kwh ~ hogar_id + grados_calor + metros_cuadrados +
    aislamiento + ocupantes + antiguedad_anios + tarifa_horaria,
  data = energia_train
) %>%
step_rm(hogar_id) %>%
step_dummy(all_nominal_predictors()) %>%
step_interact(terms = ~ metros_cuadrados:starts_with("aislamiento_"))

# Modelo + workflow + ajuste
modelo_lm <- linear_reg() %>% set_engine("lm")
wf <- workflow() %>% add_model(modelo_lm) %>% add_recipe(receta_interaccion)
ajuste <- fit(wf, data = energia_train)
```

Funciones nuevas: `step_interact()` · `extract_fit_engine()` para acceder al objeto lm subyacente.

Ejercicio 2: Ridge (calidad del aire)

Predecir PM2.5 con muchos predictores correlacionados (tráfico, NO2, CO).

```
# Receta: dummies, eliminar varianza cero, normalizar
receta_aire <- recipe(pm25_ug_m3 ~ ., data = aire_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())

# Ridge: mixture = 0
modelo_ridge <- linear_reg(penalty = tune(), mixture = 0) %>%
  set_engine("glmnet")

# Tuning con 10-fold CV
folds <- vfold_cv(aire_train, v = 10)
grilla <- grid_regular(penalty(range = c(-3, 1)), levels = 50)
tune_ridge <- tune_grid(wf_ridge, resamples = folds, grid = grilla,
  metrics = metric_set(rmse, rsq))
mejor <- select_best(tune_ridge, metric = "rmse")
ajuste_final <- finalize_workflow(wf_ridge, mejor) %>% fit(aire_train)
```

Ejercicio 3: Lasso (clínicas respiratorias)

Muchas variables candidatas, pocas relevantes. Lasso selecciona automáticamente.

```
# Partición temporal (datos en serie de tiempo)
split <- initial_time_split(clinicas, prop = 0.75)
train <- training(split) %>% select(-semana)
test  <- testing(split)  %>% select(-semana)

# Receta y modelo Lasso (mixture = 1)
receta <- recipe(consultas_respiratorias ~ ., data = train) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())

modelo_lasso <- linear_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")

# Tuning + finalización (igual que ridge)
# ...

# Ver coeficientes seleccionados (los que NO son cero)
tidy(ajuste_lasso_min) %>% filter(estimate != 0)
```

Cheatsheet: funciones por etapa

Etapa	Paquete	Funciones clave
Dividir datos	<code>rsample</code>	<code>initial_split</code> , <code>training</code> , <code>testing</code> , <code>vfold_cv</code> , <code>initial_time_split</code>
Preprocesar	<code>recipes</code>	<code>recipe</code> , <code>step_rm</code> , <code>step_dummy</code> , <code>step_zv</code> , <code>step_normalize</code> , <code>step_interact</code>
Especificar modelo	<code>parsnip</code>	<code>linear_reg</code> , <code>set_engine</code> (<code>lm</code> o <code>glmnet</code>)
Combinar	<code>workflows</code>	<code>workflow</code> , <code>add_model</code> , <code>add_recipe</code> , <code>finalize_workflow</code>
Tunear	<code>tune + dials</code>	<code>tune()</code> , <code>tune_grid</code> , <code>grid_regular</code> , <code>penalty</code>
Evaluar	<code>yardstick</code>	<code>metrics</code> , <code>metric_set</code> , <code>rmse</code> , <code>rsq</code> , <code>mae</code>
Recoger	<code>tune</code>	<code>collect_metrics</code> , <code>select_best</code>
Inspeccionar	<code>broom + workflows</code>	<code>tidy</code> , <code>extract_fit_engine</code>

Recursos para profundizar

Sitios oficiales

- tidymodels.org — documentación principal
- tidymodels.tidymodels.org — referencia API
- tmwr.org — libro 'Tidy Modeling with R' (Max Kuhn y Julia Silge)

Todo gratuito y de alta calidad.

Libro complementario (ISLR)

An Introduction to Statistical Learning,
Cap. 3, 6 y 7.

Aporta la teoría; tidymodels aporta
la implementación limpia en R.

Cheatsheets oficiales

- [rsample](#) • [recipes](#) • [parsnip](#) • [workflows](#)
- [tune](#) • [yardstick](#) • [broom](#)

Disponibles en:
posit.co/resources/cheatsheets/

Práctica recomendada

- Repetir los 3 ejercicios cambiando datos
- Probar `rand_forest()` en lugar de `linear_reg()`
- Inspeccionar `collect_metrics()` con `ggplot2`
- Usar `tidy()` para entender los coeficientes

Gracias

¿Preguntas?